

OmniSim: Simulating Hardware with C Speed and RTL Accuracy for High-Level Synthesis Designs

Rishov Sarkar
Georgia Institute of Technology
Atlanta, GA
rishov.sarkar@gatech.edu

Cong Hao
Georgia Institute of Technology
Atlanta, GA
callie.hao@gatech.edu

Abstract

High-Level Synthesis (HLS) is increasingly popular for hardware design using C/C++ instead of Register-Transfer Level (RTL). To express concurrent hardware behavior in a sequential language like C/C++, HLS tools introduce constructs such as infinite loops and dataflow modules connected by FIFOs (first-in first-out). While these constructs can *represent* concurrency, efficiently and accurately *simulating* them at C level remains challenging. First, without hardware timing information, functional verification typically requires slow RTL synthesis and simulation, as the current approaches in commercial HLS tools. Second, cycle-accurate performance metrics, such as end-to-end latency or throughput, also rely on RTL simulation. No existing HLS tool fully overcomes the first limitation. For the second, prior work such as LightningSim partially improves simulation speed but lacks support for advanced dataflow features like cyclic dependencies and non-blocking FIFO accesses.

To overcome both limitations, we propose **OmniSim**, a framework that significantly extends the simulation capabilities of both academic and commercial HLS tools. First, OmniSim enables fast and accurate simulation of complex dataflow designs, especially those explicitly declared unsupported by commercial tools. It does so through sophisticated software multi-threading, where threads are orchestrated by querying and updating a set of FIFO tables that explicitly record exact hardware timing of each FIFO access. Second, OmniSim achieves near-C simulation speed with near-RTL accuracy for both functionality and performance, via flexibly coupled and overlapped functionality and performance simulations.

We demonstrate that OmniSim successfully simulates **eleven** designs previously unsupported by any HLS tool, achieving up to **35.9×** speedup over traditional C/RTL co-simulation, and up to **6.61×** speedup over the state-of-the-art yet less capable simulator, LightningSim, on its own benchmark suite.

Keywords

Dataflow Designs, Design Simulation, High-Level Synthesis

ACM Reference Format:

Rishov Sarkar and Cong Hao. 2025. OmniSim: Simulating Hardware with C Speed and RTL Accuracy for High-Level Synthesis Designs. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3725843.3756033>



This work is licensed under a Creative Commons Attribution 4.0 International License. *MICRO '25, Seoul, Republic of Korea*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1573-0/2025/10

<https://doi.org/10.1145/3725843.3756033>

1 Introduction

► **Hardware design using HLS.** High-Level Synthesis (HLS) has gained popularity in hardware design due to its significantly improved productivity, allowing users to develop hardware using high-level languages such as C/C++, which will be automatically synthesized into Register Transfer Level (RTL) code. When designing hardware, users typically care about two key aspects: (1) whether the design behaves functionally as expected, known as **functionality simulation**, and (2) how well the synthesized hardware performs in terms of latency or throughput, known as **performance simulation**.¹

As illustrated in Fig. 1, a typical HLS flow consists of three stages: C simulation, C synthesis, and C/RTL co-simulation. C/RTL co-simulation offers the highest fidelity with cycle-level validation, but is notoriously slow, often requiring hours or days, undermining the productivity benefits of HLS. At the other extreme, C simulation is fast and lightweight, but provides only limited functional validation and no performance insights. C synthesis lies in between: after generating RTL, the tool produces static performance estimates, which unfortunately are often inaccurate or even unavailable, especially with variable loop bounds or complex control flow.

While much recent work, such as Verilator [1] and FireSim [12], focuses on speeding up RTL simulation, our focus is on an earlier point in the design cycle: *simulating HLS designs before RTL is even generated, for both functionality and performance*. Our goal is to deliver RTL-level accuracy at near-C simulation speed, enabling rapid design-space exploration without waiting for RTL generation.

► **Challenges of C-level simulation.** Current HLS tools can simulate functionality at the C level for *some* designs, but their performance estimation is often inaccurate due to the lack of runtime details such as actual loop counts, branch behavior, and FIFO stalls [2, 9, 19, 20]. The challenges become greater for *more complex designs*. To express concurrent hardware behavior in sequential C/C++, HLS tools rely on special constructs such as infinite loops and dataflow (streaming) modules connected by non-blocking FIFOs. These constructs can *represent* a wide range of hardware designs, but they make it particularly difficult to correctly *simulate* them at the C level, both for functionality and for performance.

► **Functionality simulation.** Certain common RTL scenarios are poorly supported by C-level simulation. For example, a timer module that tracks the cycle count of another module, or a network router that dynamically changes output ports depending on congestion, must be implemented as **dataflow designs** with *non-blocking reads and writes* (see Sec. 2). However, even commercial tools such as Vitis HLS [5] and Catapult HLS [21] struggle to simulate such

¹Also referred to as functionality verification, timing simulation, or performance testing by different HLS tool vendors.

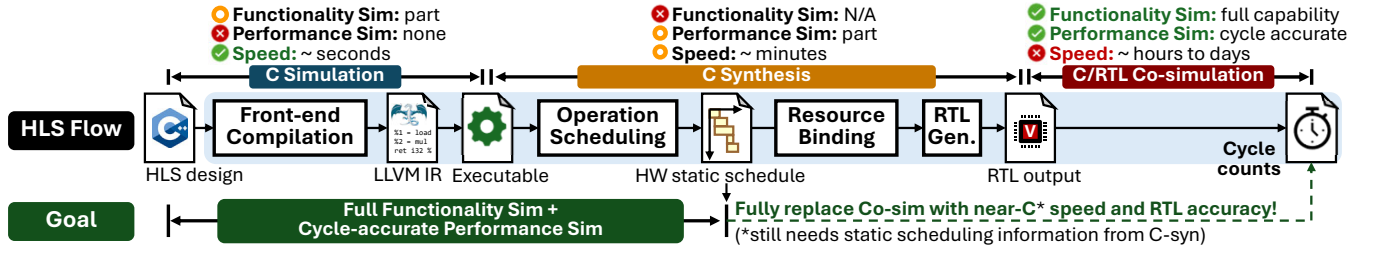


Figure 1: Overview of a representative HLS flow and the goal of this work.

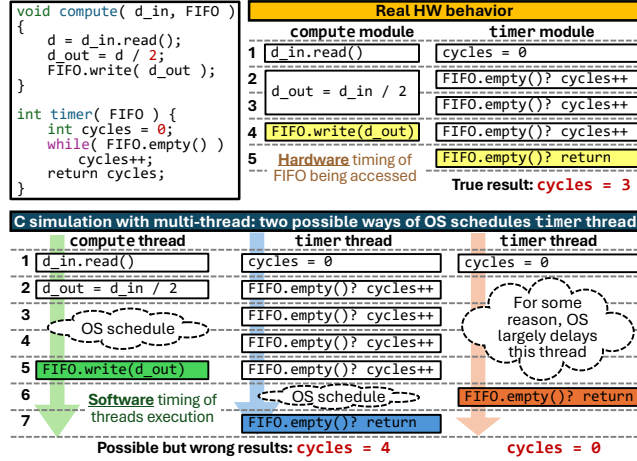


Figure 2: Accurately simulating hardware functionality and performance at C level is highly challenging due to the mismatch between sequential C and concurrent hardware: key actions (e.g., FIFO access) must be captured by their hardware timing, not software timing from thread scheduling.

behavior correctly. Their manuals explicitly warn that C-level simulation may produce incorrect behavior, forcing users to rely on RTL simulation, which is both slow and tedious.^{2,3}

2 Performance simulation. Beyond functionality simulation, accurately predicting performance, such as cycle counts or throughput, at the C level is even more challenging. This is because performance depends not only on correct functional behavior (already difficult to achieve at this level) *but also* on hardware-specific details such as scheduling, pipelining, and finite-state machine (FSM) transitions. Similarly, commercial HLS tools lack satisfactory solutions, either providing rough and inaccurate performance estimates after full synthesis, or fall back to slow RTL simulation.^{4,5}

²Catapult manual: “If the top level of your design contains non-blocking IO, the tool may not produce results as expected.” “While the synthesis of a non-blocking read is trivial, the resulting hardware will rarely, if ever, match the functionality of the original C code; all testing done at the C level would need to be re-done in the RTL.”

³Vitis HLS manual: “Using non-blocking APIs can lead to non-deterministic behavior which cannot be fully validated during C-simulation and requires an RTL test bench to test it exhaustively.”

⁴Catapult manual: “Non-blocking IO cannot be used for throughput-accurate simulation.” “Latency testing using the source code is not commonly done (and is not even possible in C++-based designs), so there is no best practice for this yet.”

⁵Vitis HLS manual: “During C simulation, streams have an infinite size. It is therefore not possible to validate with C simulation, and these methods can be verified only during RTL simulation when the FIFO sizes are defined.”

3 Tightly coupled functionality and performance. The two challenges above both arise from the semantic gap between un-timed, sequential C/C++ and the timed, concurrent nature of hardware. This gap creates a fundamental mismatch between *hardware-level timing of actions* and the *OS-controlled scheduling of software threads* during simulation.

To model concurrent hardware at the C level, particularly designs with multiple active modules with cyclic dependencies and/or infinite loops, it is natural to use C/C++ multi-threading to mimic simultaneous activity. However, in naive multi-threaded simulation, execution order is dictated entirely by the OS scheduler, which can distort timing and compromise correctness.

Fig. 2 illustrates this with a simple example: a timer module monitors a compute module by polling a FIFO for results. The expected cycles value is three based on *hardware timing*. However, if simulated naively with software threads, the timing of FIFO accesses is governed by the OS’s scheduling—i.e., *software timing*—leading to incorrect results, as shown in the two cases in Fig.2.

This example highlights two key implications. First, correct simulation must preserve *true hardware timing*, independent of OS scheduling, by explicitly orchestrating thread execution and tracking cycles (e.g., ensuring accurate FIFO full/empty status at every cycle). Second, in many dataflow designs, correct functionality depends directly on correct timing, making functionality and performance simulation inherently inseparable and tightly coupled.

► Existing work. For complex dataflow designs (e.g., with non-blocking FIFO access) like the one presented in our motivating example, existing HLS tools—including commercial ones—fail to correctly simulate functionality, let alone performance. For non-dataflow designs or simpler ones, while functionality simulation is often straightforward, achieving fast yet cycle-accurate performance simulation remains a significant challenge. Various tools such as FLASH [9], FastSim [2], and LightningSim [19, 20] have been developed as faster alternatives to RTL simulation, but they support only a limited subset of HLS designs.

► Our approach. To address the limitations of existing HLS simulation, we propose OmniSim to: (1) **extend C-level simulation capability of HLS tools** by enabling both functionality and performance simulations for those *complex dataflow designs* that are currently unsupported or considered infeasible; (2) **achieve near-C speed while maintaining RTL accuracy**, for both simple and complex designs. We propose the below key technologies:

• **Sophisticated orchestration for software multi-threading.** First, to simulate concurrent behaviors of multiple active hardware modules, especially those with cyclic dependencies, we

propose software-based multi-threading, where the functionality of each active module is simulated by one software thread. Second, to align software thread execution with actual hardware timing, we introduce a dedicated performance simulation thread with an orchestration mechanism among the threads, including: thread requests, queries, and FIFO tables, to accurately track hardware-level timing despite arbitrary OS scheduling. This ensures functional correctness that precisely matches RTL behavior—something commercial tools explicitly declare infeasible.

- **Flexibly coupled and overlapped functionality and performance simulation.** Through multi-thread orchestration, OmniSim flexibly separates and re-integrates functionality and performance simulation on demand. This is not only essential to expanding HLS simulation capability, but the overlapped execution also delivers significant speedup compared to the state-of-the-art (but less capable) simulator, LightningSim.

Other contributions are highlighted as follows:

- **Taxonomy and classification of dataflow designs.** We present the first systematic, in-depth analysis of dataflow designs to determine whether they can be naively simulated using standard C/C++. We categorize them into **Type A, B, and C**, and explain the fundamental simulation challenges.
- **Deadlock detection and incremental simulation.** OmniSim includes a crucial feature to detect deadlocks immediately as they occur, which are common in dataflow designs, without causing the simulation to hang. It is also able to maximally reuse previous simulation progress when FIFO sizes are modified, avoiding a complete restart and saving simulation time.
- **Evaluation.** We design a benchmark suite consisting of eleven C programs that cannot be correctly simulated at C-level by previous HLS tools. All of them can now be successfully simulated by OmniSim while being $30.7\times$ faster than C/RTL co-simulation. We also demonstrate superior performance over the state-of-the-art HLS simulator, achieving up to $6.61\times$ speedup. Furthermore, the incremental simulation capability of OmniSim shows *four orders of magnitude* speedup over full simulation.

2 Background

2.1 High-Level Synthesis Simulation

As depicted in Fig. 1, most HLS tools (using Vitis HLS as an example) have three stages, where the output of each stage presents clear tradeoffs in terms of functionality and performance simulation:

- **C Simulation (C-sim).** HLS front-end compiles the source code into intermediate representation (IR), such as LLVM IR, and executes a standard C/C++ binary to verify functional correctness. *C-sim executes in seconds and enables partial functional simulation, but it lacks hardware semantics and provides no performance estimates.* In addition, as already shown in Fig. 2 (Sec. 1), existing C-sim cannot correctly simulate complex dataflow designs.
- **C Synthesis (C-syn).** After scheduling and binding, finite state machines (FSMs) are generated to represent control flow, and operations are scheduled and bound to specific hardware resources. The output is a static hardware schedule that captures module dependencies, concurrency, and pipeline parallelism. *C-syn provides a rough estimate of static latency but does not model dynamic behaviors (e.g., non-blocking) and does not validate functionality.*

- **C/RTL Co-simulation (Co-sim).** After RTL generation, testbenches are also generated to enable cycle-accurate simulation. Co-sim⁶ verifies both functional correctness and performance, reporting detailed hardware performance in terms of cycle counts. *However, it is significantly slower—often requiring hours or even days to simulate a single testbench.*

2.2 Dataflow Designs

Dataflow designs are prevalent in many domain-specific accelerators [3, 6, 11, 14–17], enabling tasks to execute concurrently and communicate via FIFO channels for high performance. Beyond accelerators, they are also effective in representing real-world hardware designs such as multi-core, out-of-order, and network switching/routing. In HLS, users can define parallel tasks as functions, which are synthesized into hardware modules connected by FIFOs.

2.2.1 FIFO Access: Blocking and Non-Blocking. FIFO streams have two access patterns: *blocking* (B) and *non-blocking* (NB). Blocking accesses either complete immediately if the operation is feasible or stall the task until it becomes so. For example, a task writing to a full FIFO will stall until space becomes available (i.e., an element is consumed), and similar for a read operation from an empty FIFO.

In contrast, non-blocking accesses never stall the hardware, but simply return a Boolean value indicating the success or failure of the read/write operation. For instance, a non-blocking read (`read_nb()` in Vitis HLS) returns `true` and pops an element if the FIFO is not empty; otherwise, it returns `false` immediately without waiting. Methods such as `empty()` and `full()` are also commonly used to inspect FIFO status, typically in conjunction with `if-else` branches.

Non-blocking access is critical in many real-world applications. For example, a real-time video processor must handle frames as they arrive; a non-blocking pipeline allows frames to be dropped under heavy load, avoiding backpressure and enabling timely processing of subsequent frames. Similarly, in network switches, non-blocking packet handling enables dynamic rerouting when certain processing paths are congested. More broadly, non-blocking behavior is fundamental to architectural features such as out-of-order CPU execution and non-blocking cache miss handling. However, despite its importance, simulating non-blocking behavior at the C level is challenging and generally unsupported by commercial HLS tools.

2.2.2 Dataflow Graphs: Acyclic and Cyclic. When connected via FIFOs, dataflow tasks may form either acyclic or cyclic dependencies. In cyclic designs, an earlier-defined task in C may expect inputs from a later-defined task through a FIFO, creating a feedback loop. This significantly complicates simulation: sequential execution of C functions becomes invalid, as a function waiting for input from a subsequent module would stall indefinitely. Accurately simulating such systems requires modeling hardware concurrency using multi-threading—a non-trivial task, as illustrated in our earlier example.

3 Simulating Dataflow Designs at C Level

We introduce a detailed analysis and categorization of dataflow designs, highlighting their specific C-level simulation challenges. To the best of our knowledge, this is the first comprehensive study to propose a well-defined taxonomy and classification of such designs.

⁶Terminology in Vitis HLS; Catapult uses SCVerify with similar capabilities.

Design Type	Defining Features			Simulation Requirement		Supported by
	Module Dependency	Dataflow Type	Program Behaviors	Func Sim	Perf Sim	
Type A	Acyclic only	Non-dataflow or Blocking only	One	L1	L1	LightningSim
Type B	Cyclic (or acyclic)	Non-blocking (with Blocking)	One, regardless of NB	L2	L3	OmniSim
Type C	Cyclic (or acyclic)	Non-blocking (with Blocking)	Multiple, depending on NB	L3	L3	
Feature Implications	Func Sim and Perf Sim both become concurrency-dependent	+ Perf Sim becomes cycle-dependent (exact cycles needed) +	Func Sim also becomes cycle-dependent thus must depend on Perf Sim	➡ Func Sim and Perf Sim must be coupled and run simultaneously using multi-threaded		

Figure 3: Defining features of three types of dataflow designs with implications on performance and functionality simulation.

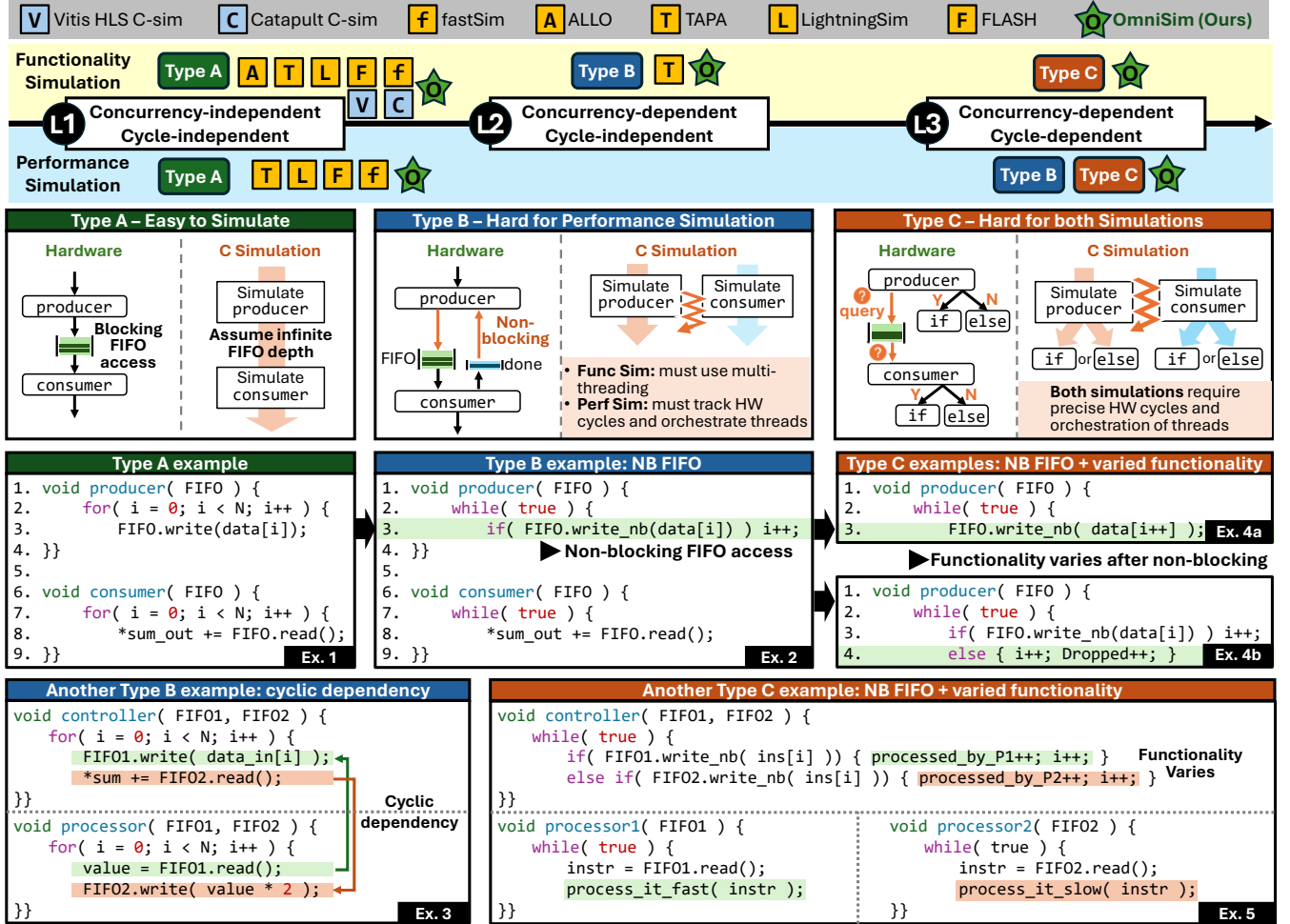


Figure 4: Taxonomy, classification, and examples of dataflow designs.

3.1 Taxonomy and Design Classification

As shown in Fig. 3, we classify dataflow designs into three types based on three defining features: module dependency, FIFO access type, and the number of possible program behaviors after FIFO access. As will be detailed in Sec. 3.2, each feature has specific implications for both functionality simulation (Func Sim) and performance simulation (Perf Sim), resulting in different levels of complexity. Supporting these defining features and addressing their simulation implications are the key motivations behind OmniSim, fundamentally distinguishing it from LightningSim.

- Type A:** Designs that are either non-dataflow or use only *blocking* FIFO accesses. Modules must have strictly *acyclic* dependencies, and there is only *one* possible program behavior after any FIFO access. Example. Fig. 4 Ex. 1 shows a basic setup where a producer sends data to a consumer via a blocking FIFO. Practical Usage. Type A is typical in fixed-function accelerators such as systolic arrays, convolution engines, and digital signal processing (DSP) pipelines, where data moves in a predictable, lockstep manner.
- Type B:** Designs that may use *non-blocking* FIFO accesses, include *infinite* loops, or contain *cyclic dependencies*, yet still exhibit

only *one* possible program behavior per FIFO access. That is, the success or failure of NB accesses *does not alter* the program behavior. Examples. Ex. 2 extends Ex. 1 with NB FIFO accesses and infinite loops, modeling a polling-based stream processor. Ex. 3 introduces a cyclic dependency between a controller and a processor, communicating via blocking FIFOs. Practical Usage. Type B designs are common to describe systems that respond to asynchronous or event-driven inputs without data-dependent branching. Examples include DMA engines polling memory-mapped FIFOs, stream filters that process packets as they arrive, or FSMs that iterate through actions. Cyclic dependencies are also common in systems with instructions and controllers, where feedback loops coordinate state updates and actuation.

- **Type C:** Designs that exhibit Type B characteristics but allow *multiple possible behaviors* following a NB FIFO access. Such variability often arises from explicit control branches, data drops, or internal state updates in response to FIFO access outcomes. Examples. Ex. 4a and Ex. 4b modify Ex. 2 to introduce divergent behaviors when a write fails: In Ex. 4a, data is silently dropped (`i++` still executes); In Ex. 4b, an `if-else` branch tracks failures explicitly. Ex. 5 shows a controller dispatching to one of two processors depending on which is less busy, dynamically responding to system state. Practical Usage. Type C reflects the complexity of real-world hardware systems that adapt to dynamic resource, timing, or data conditions. Examples include out-of-order execution engines, adaptive packet routers, cache controllers handling hits/misses, dynamic memory allocators, and streaming accelerators that repartition workloads under backpressure.

3.2 Implications on Simulation Requirements

The defining features of dataflow designs directly determine their simulation requirements. We identify two key considerations for accurate simulation: **(1) hardware concurrency (in)dependence** — whether the simulation must model concurrent module execution (e.g., via multi-threading) to reflect simultaneous activity, and **(2) hardware cycle (in)dependence** — whether the simulation must capture the exact clock cycle at which hardware actions, such as FIFO reads and writes, occur.

Accordingly, there are three valid levels of simulation requirements, shown in the top row of Fig. 4: **L1** *concurrency- and cycle-independent*, **L2** *concurrency-dependent, cycle-independent*, and **L3** *concurrency- and cycle-dependent*. Only three combinations are possible, as cycle-dependence always implies concurrency-dependence.

The three defining features of design types map to these levels as annotated at the bottom of Fig. 3. First, designs with *cyclic dependencies* require both functional and performance simulation to be concurrency-dependent, necessitating multi-threading. Second, designs with *NB FIFO accesses* require performance simulation to be cycle-dependent, where the timing of each FIFO access must be captured dynamically rather than determined statically. Third, when the outcome of an NB access can change subsequent behavior, functional simulation also becomes cycle-dependent, because the functional correctness now depends on exact cycles.

These implications underscore the need for **tightly coupled functional and performance simulation executed concurrently via multi-threading**, which is the central goal of OmniSim.

3.2.1 Type A: Func Sim Level 1, Perf Sim Level 1.

Type A designs can be simulated sequentially using a single thread for both Func Sim and Perf Sim. Although the hardware executes modules concurrently, *multi-threading is unnecessary* because FIFO access order is deterministic under the assumption of infinite FIFO depth. Consequently, both simulations are concurrency-independent and can be fully decoupled, as functional correctness does not depend on precise hardware cycles, and hardware cycle counts can be calculated statically after the fact based on Func Sim.

3.2.2 Type B: Func Sim Level 2, Perf Sim Level 3.

► **Functionality simulation.** Type B designs are *concurrency-dependent* for Func Sim, meaning multi-threading is required. This occurs when modules contain infinite loops that must execute in parallel. For example, in Ex. 2, if the `while` loop in the producer is simulated sequentially, any module defined after it (e.g., consumer) would never run. Similarly, in Ex. 3, which contains cyclic dependencies, the controller may wait for input from the processor before proceeding. Even without infinite loops, such dependencies require concurrent simulation.

However, Type B designs are *cycle-independent* for Func Sim, because the functionality correctness depends only on *what* hardware actions occur (e.g., FIFO accesses), not on *when* they occur. In Ex. 2, as long as the producer’s behavior after a successful NB write is correctly modeled, the exact cycle of the write, whether at cycle 5 or cycle 10, is irrelevant. In Ex. 3, although cyclic dependencies require multi-threading, blocking accesses ensure a single valid execution order, which can be enforced with multi-thread locks and thus no extra orchestration is needed. In short, even if Perf Sim is inaccurate, Func Sim can still be correct for Type B designs.

► **Performance simulation.** Unlike Func Sim, performance simulation of Type B designs is *cycle-dependent* due to NB FIFO accesses. For example, in Ex. 2, suppose we measure the number of cycles required for the producer to perform five successful FIFO writes (break condition omitted). One OS thread simulates the producer and another simulates the consumer. If the OS scheduler happens to run the producer thread significantly earlier or more often than the consumer thread, the simulation may record extra failed write attempts before the consumer has time to read from the FIFO, leading to an overestimated cycle count.

3.2.3 Type C: Func Sim Level 3, Perf Sim Level 3.

Type C designs are concurrency- and cycle-dependent for both Func Sim and Perf Sim. The simulator must capture not only *what* actions occur but also *when* they occur, since functional behavior can change depending on the outcome of a NB FIFO access. For example, in Fig. 2, the computed value of `cycles` may vary entirely due to OS-level thread scheduling rather than true hardware timing. Similarly, in Ex. 4 and Ex. 5, omitting the exact cycle of a FIFO access can lead to both functional errors and inaccurate performance estimates. Accurate simulation therefore requires precise modeling of *actual hardware cycles* rather than relying on *software-level timing* dictated by the OS, making Level 3 simulation fundamentally challenging.

3.2.4 Additional Challenge: Deadlock Detection.

For Type B and Type C designs, a further challenge is detecting hardware-level deadlocks originated from flawed design logic. For

example, if a producer waits for a consumer while the consumer simultaneously waits for the producer, the system can deadlock regardless of FIFO depth. A simulator must distinguish such deadlocks from temporary stalls and report them promptly to avoid indefinite hangs where all modules are idle. This differs from live-lock, where modules remain active but make no forward progress; neither OmniSim nor C/RTL co-simulation detects livelocks.

4 Prior Work

Catapult and Vitis HLS are two representative commercial HLS tools, while LegUp [7] and Bambu [10] are open-source academic alternatives. Among recent academic efforts, TAPA [11] is a specialized tool designed to support multi-kernel dataflow designs using C++, and Allo [8] is a Python-based HLS framework that generates Vitis-compatible C programs. In addition to these end-to-end HLS tools, several studies have focused on accelerating performance simulation at the C level, including FLASH [9], FastSim [2], HLPef [25], and LightningSim [19, 20].

► **Functionality simulation.** All existing HLS tools, both commercial and academic, can simulate only certain types of dataflow designs at the C level, specifically, Type A in our taxonomy. As discussed in Sec. 1, both Catapult and Vitis impose explicit restrictions on simulating non-blocking designs (Type B and C). Among academic tools, TAPA can simulate Type A and B but not C, while Allo supports only Type A. For unsupported types, these tools either disallow or fall back to RTL simulation to ensure correctness.

► **Performance simulation.** To perform fast performance simulation, FLASH uses C++ to model the timing of HLS designs by combining the original source code with scheduling information from the HLS tool. FastSim models the generated Verilog using C and applies optimizations tailored to common HLS structures, such as FSMs. HLPef, designed for modeling GNN accelerators, prioritizes simulation speed over cycle accuracy by omitting full functional simulation. LightningSim integrates both functionality and timing simulation at the LLVM IR level by injecting hardware scheduling information into the IR.

For simple Type A designs, all of these tools demonstrate reasonably good performance and accuracy. Among them, LightningSim stands out as both the fastest and most accurate, achieving near-C execution speed with 99.9% RTL-level accuracy. However, for more complex Type B and Type C designs, none of the previous tools are able to simulate correctly.

5 OmniSim Overview

Since OmniSim is inspired by LightningSim, we begin by comparing the two simulators and highlighting their key differences, followed by a simplified walk-through example that demonstrates the core techniques of OmniSim.

Fig. 5 illustrates the key differences between LightningSim and OmniSim. LightningSim adopts a two-stage simulation model that fully decouples functionality simulation from performance simulation. This approach is effective for Type A designs, but is fundamentally limiting for Type B and C designs: as discussed in Sec. 3, these design types require precise hardware timing information *during* functionality simulation, which cannot be obtained in a decoupled simulation workflow.

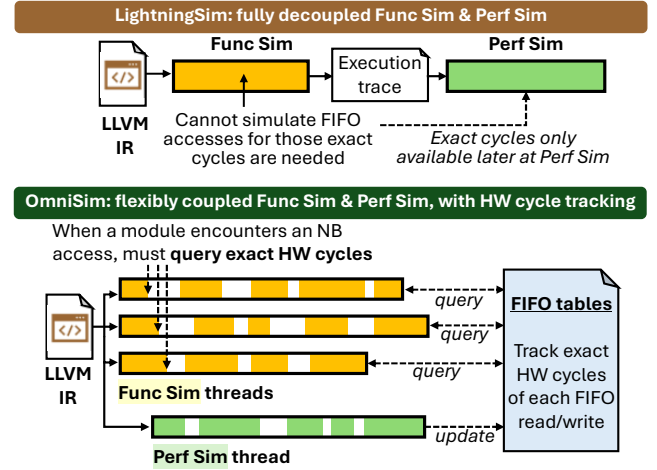


Figure 5: Comparing LightningSim and OmniSim.

In contrast, OmniSim introduces a fundamentally different simulation paradigm supported by novel data structures. First, it tightly couples functionality and performance simulation by coordinating multiple functionality simulation threads with a centralized performance simulation thread. The performance thread provides exact timing information to the functionality threads on demand. Second, to ensure correct hardware timing regardless of arbitrary OS-level thread scheduling, OmniSim employs a robust thread orchestration mechanism synchronized via a set of FIFO timing tables. These tables dynamically track precise hardware cycle information for each FIFO access. Functionality threads query the tables as needed, while the performance thread updates them in real time. Together, these two core techniques allow OmniSim to support all types of dataflow designs, making it significantly more general-purpose and capable than LightningSim.

5.1 LightningSim: Decoupled Two Phases

Given a simple producer-consumer example with only blocking (B) FIFO accesses, LightningSim uses a single thread to simulate both modules sequentially. As illustrated in Fig. 6 (top), LightningSim operates in two distinct phases: *untimed* functionality is simulated in Phase 1 through trace generation without hardware constraints, and *timed* performance is simulated in Phase 2 via trace analysis.

► **Phase 1: Trace and Simulation Graph Generation (untimed).** Given an HLS design and testbench, LightningSim extracts the LLVM IR after front-end compilation, instruments the IR for the host platform (for example, x86), and executes it to record a trace of the executed basic blocks. This phase performs functional simulation and generates the trace data required for performance modeling.

Next, it uses the *static schedule* for each module—generated during C synthesis—to construct *dynamic stages*. These stages are derived by “unrolling” the static schedule based on the execution trace. For example, if a loop has a static schedule of 1 and is executed four times with an initiation interval of one, the resulting dynamic stages will be labeled 1, 2, 3, and 4.

As shown in Fig. 6, the output of Phase 1 includes **event lists** and a **simulation graph**. An *event* refers to an operation that may potentially cause a *stall*, such as a FIFO access or a function call.

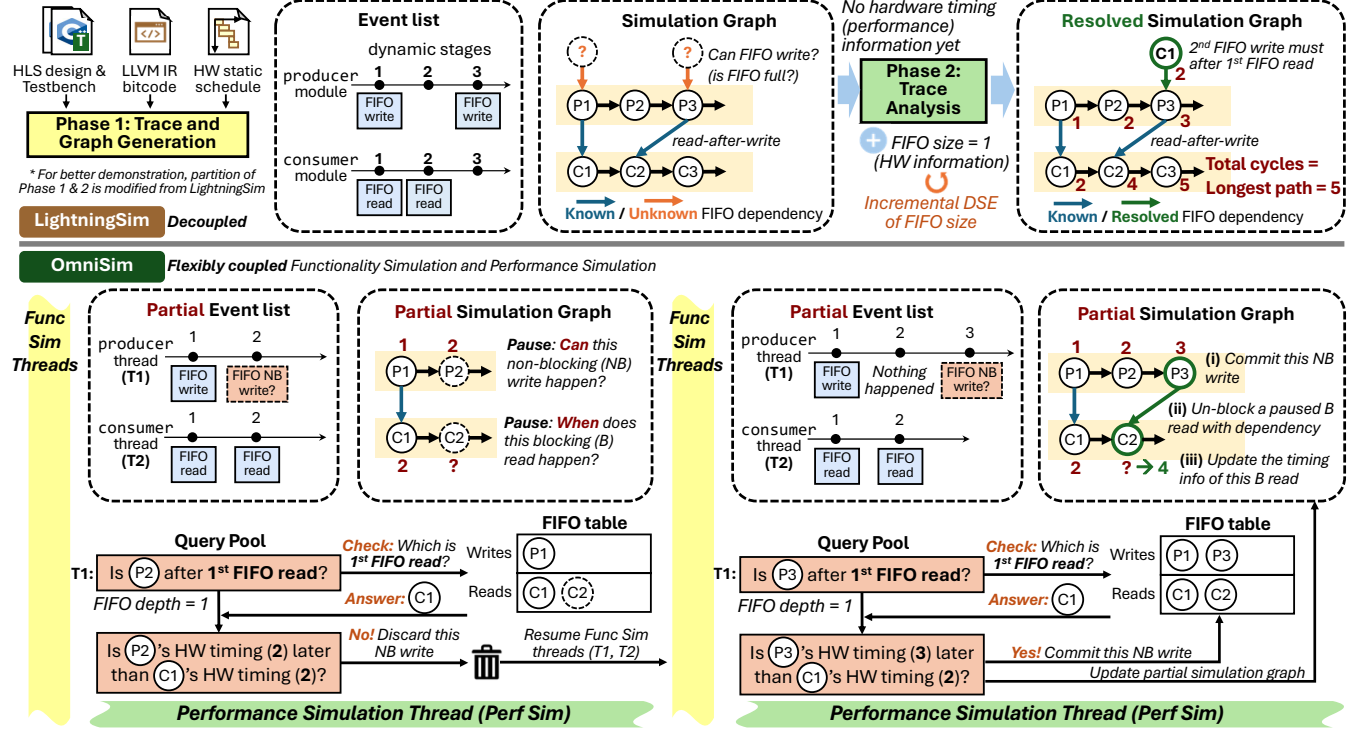


Figure 6: Overview of the OmniSim workflow compared to LightningSim. LightningSim executes functionality simulation and performance simulation as two fully decoupled phases. In contrast, OmniSim tightly integrates these two simulations, allowing flexible overlap to improve efficiency. The overlapping execution is illustrated in detail in Fig. 7.

Each module maintains an event list aligned with its dynamic stages, where specific stages are associated with events. A simulation graph is then constructed to capture dependencies among events, including function calls and FIFO read-after-write constraints. For instance, in the example shown, the consumer's first read (C1) must follow the producer's first write (P1), resulting in a directed edge from P1 to C1. Similarly, P3 precedes C2.

However, certain dependencies, such as FIFO write-after-read, remain unresolved at this stage. A module cannot write to a FIFO if it is full, but the simulator cannot determine fullness without knowing the FIFO's capacity. These unresolved dependencies are marked as *unknown* and are addressed in the next phase.

► **Phase 2: Trace Analysis (timed with HW info).** In this phase, LightningSim incorporates hardware-specific constraints, such as FIFO sizes, to perform *stall analysis* and resolve the previously unknown dependencies. In the same example, suppose the FIFO has a depth of one. The first write operation (P1) can proceed immediately, but the second write (P3) must wait until the first read (C1) completes. This dependency is enforced by adding an edge from C1 to P3, indicating that P3's write must occur after C1's read. Finally, LightningSim computes the total execution latency by identifying the longest path from the start node to all other nodes in the simulation graph, resulting in a cycle-accurate performance simulation of the original HLS design. In this example, the total cycle count is 5, where node C2 is stalled by one cycle from P3.

5.2 OmniSim: Interleaved Two Phases

We now modify the example in Fig. 6 (bottom) by introducing a non-blocking (NB) FIFO write at dynamic stage 2 of the producer module. This changes the design from Type A into Type B or C, requiring a fundamentally different simulation strategy.

First, as discussed in Sec. 3, multi-threading is now required to simulate both modules. OmniSim assigns one thread per dataflow module: thread T1 simulates the producer, and thread T2 simulates the consumer. These are referred to as **Func Sim threads**.

Second, the simulator must determine whether the FIFO is full precisely at the *hardware cycle* when a NB FIFO write is issued (similarly for NB reads). This determination depends on how many reads from T2 have occurred up to that cycle—not on the OS scheduling order of T1 and T2. To resolve this, OmniSim introduces a dedicated **Perf Sim thread**, responsible for synchronizing NB accesses and maintaining FIFO status based on hardware timing.

► **Thread orchestration and example walk-through.** The threads must be carefully orchestrated. First, during simulation, each Func Sim thread proceeds independently, computes its own hardware timing (cycle count) accurately, just like LightningSim, until it encounters a NB FIFO event. In this example, T1 and T2 each complete one blocking FIFO access, resulting in nodes P1 and C1 in the **partial simulation graph**, with a dependency from P1 to C1. Accordingly, P1 is assigned cycle 1 and C1 is assigned cycle 2.

Second, T1 reaches a NB FIFO write at dynamic stage 2, denoted as P2, pauses, and issues a **query**: will this FIFO write succeed? This will be answered by the Perf Sim thread. Meanwhile, T2 also pauses, waiting on a blocking FIFO read because data is unavailable.

Third, once both Func Sim threads are paused, the Perf Sim thread becomes active to attempt to answer all queries. It maintains a **FIFO read/write table** to record all committed and pending accesses along with their associated hardware cycles. Instead of using a simple FIFO occupancy counter, this table is essential as it accurately tracks event timing despite the misalignment between software thread scheduling and actual hardware behavior.

In our example, assume the FIFO has a depth of one. The query from T1—“Can the NB write at cycle 2 succeed?”—is translated into: “Has the first FIFO read completed before this write?” If not, the FIFO is still full, and the write must fail. The Perf Sim thread consults the FIFO table, confirming that the first FIFO read occurred at node C1, which is also at cycle 2. The query is further turned into: “Is P2 (cycle 2) strictly after C1 (cycle 2)?” The answer is no. Therefore, the NB write at P2 fails and is discarded. No state change occurs at cycle 2, and the Func Sim threads resume execution.

Fourth, at dynamic stage 3, T1 attempts another NB FIFO write, labeled P3. The new query is: “Is P3 (cycle 3) after C1 (cycle 2)?” This time, the answer is yes. As a result: (i) the NB write is committed by adding P3; (ii) the pending blocking read at T2 is unblocked by establishing a dependency from P3 to C2; and (iii) the simulation graph is updated with C2 assigned to cycle 4, indicating that the read is satisfied immediately following the successful write.

Finally, the Func Sim and Perf Sim threads continue in this interleaved fashion until the simulation completes, yielding a final simulation graph with accurate dependencies and cycle counts.

6 OmniSim Implementation

Building on the simplified overview of thread orchestration in Sec. 5, demonstrating its support for all types of dataflow designs, we now present the detailed implementation. OmniSim consists of two stages, front-end compilation, and multi-thread execution. In the second stage, we particularly focus on a key technique, where functionality and performance simulations are not only tightly, but also *flexibly overlapped*, to improve the simulation efficiency.

6.1 Front-end Compilation

The compilation process in OmniSim requires three sets of inputs. From the user-provided HLS project, we extract (1) the LLVM IR bitcode generated by the HLS front-end compilation, and (2) the user’s testbench code, typically in C++, which invokes the HLS kernel with appropriate inputs and checks its outputs. Additionally, (3) we introduce our own runtime library, implemented as a Linux shared object (.so) file, which provides core simulation infrastructure such as FIFO buffers, AXI interfaces, and trace collection utilities. Anytime the HLS code invokes a hardware-level intrinsic such as FIFO accesses, it calls into this library, which passes FIFO data between modules and enables OmniSim to track them. However, just intercepting hardware intrinsics is not enough to correctly simulate all hardware behaviors—for instance, OmniSim must also make dataflow tasks run concurrently. In such cases, we must instead *rewrite* the HLS user’s code at the LLVM level.

Request type	Description	Query?
TraceBlock	A basic block was executed	○
StartTask	A dataflow task started in a new thread	○
FifoRead/Write	FIFO was read from/written to	○
AxiReadReq/WriteReq	A read/write request issued on AXI	○
AxiRead/Write	AXI was read from/written to	○
AxiWriteResp	A write response was issued on AXI	○
FifoCanRead/Write	Query for FIFO empty/full	●
FifoNbRead	An NB FIFO read attempted	●
FifoNbWrite	An NB FIFO write attempted	●

Table 1: Key requests made by Func Sim threads.

6.1.1 LLVM Passes. The first step in the front-end is to apply a series of custom LLVM passes to the LLVM IR, preparing it for simulation. Similar to LightningSim, we insert calls to a tracing function at the beginning of each basic block to enable fine-grained tracking of runtime execution paths at the LLVM instruction level. In addition, OmniSim introduces new LLVM passes to enhance simulation capability and performance, specifically, to rewrite dataflows to use threads, enabling tasks to execute concurrently as discussed in Sec. 3. To do this, we implement an LLVM pass that identifies dataflow functions in the IR, extracts their constituent sub-tasks, and generates a dedicated wrapper function for each sub-task. Instead of invoking tasks directly, the original dataflow function is rewritten to launch a separate thread for each wrapper function and wait for all threads to complete before returning.

6.1.2 Compiling and Linking. After applying the custom LLVM passes, we compile the transformed LLVM IR into object code using clang. The testbench C/C++ code is also compiled at this stage. The resulting objects are linked together with our runtime shared library, which provides the runtime support for FIFO and AXI interface implementations and the logic for trace collection and analysis. The final output is a binary executable that runs the simulation.

6.2 Multi-thread Execution

The execution of OmniSim consists of five major steps, annotated as ❶ to ❺ in Fig. 7. Six key data structures are maintained during the simulation, labeled as (A) to (F).

❶ Invoke all threads—Func Sim and Perf Sim. At the beginning of execution, all necessary threads are launched, including one Perf Sim thread and multiple Func Sim threads. All dataflow modules, including submodules, will be allocated a dedicated thread; modules with only blocking accesses are also assigned a thread to support cyclic dependencies and infinite loops. In Fig. 7, Func Sim threads are labeled T1 through T4.

Once initiated, Func Sim threads begin simulating their respective modules and generating **requests**, which are inserted into a global (A) **Request Queue**. Table 1 lists key (non-exhaustive) request types. Most are informative, such as blocking FIFO operations (e.g., `read()`, `write()`), and are used to update the simulation graph state, while NB accesses will result in **queries** that require resolution later, as listed in the last three rows of the table.

❷ Process all requests—by Perf Sim thread. While Func Sim threads are running, the Perf Sim thread operates concurrently, actively processing requests from the queue. As illustrated in Fig. 7, yellow highlights indicate active periods of Func Sim threads, and green highlights indicate the active periods of the Perf Sim thread.

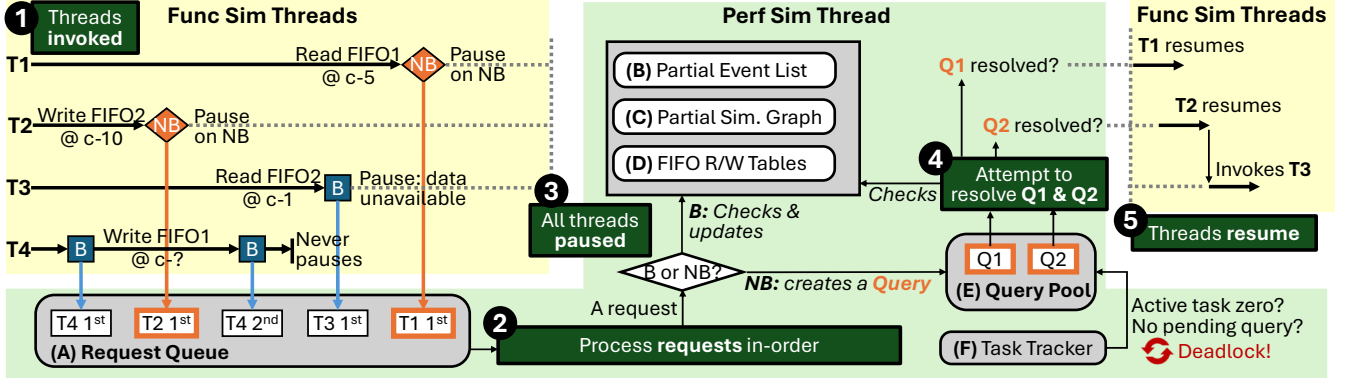


Figure 7: Execution workflow of OmniSim. Func Sim threads (yellow background) and the Perf Sim thread (green background) run concurrently with overlaps. Key steps include invoking a thread for each dataflow module, sending requests and pausing on queries, the Perf Sim thread processing requests and creating/resolving queries, and resuming Func Sim threads.

Notably, the overlap between these periods enables OmniSim to not only support Type B and C dataflow designs but also to simulate Type A designs more efficiently than LightningSim, as will be demonstrated in the evaluation.

As soon as Func Sim threads begin inserting requests into the request queue, the Perf Sim thread starts processing them. It checks and updates key data structures: (B) partial event list, (C) partial simulation graph, and (D) FIFO read/write tables. In addition, for a request that is also a NB access, the Perf Sim thread creates a **query** and places it into the (E) **Query Pool** for resolution.

3 All Func Sim threads pause. A Func Sim thread pauses whenever it encounters a query that must be answered, such as a NB read/write or FIFO status check. For example, T1 pauses at a NB read from FIFO 1 at hardware cycle c-5, since it needs to determine whether the FIFO is non-empty at that moment. Likewise, T2 pauses on a NB write to FIFO 2 at c-10 to determine whether the FIFO is full. This example again illustrates that hardware timing and software thread execution order may not align—e.g., T1 simulating cycle c-5 may be scheduled after T2 simulating cycle c-10.

Func Sim threads may also pause on blocking reads if the corresponding FIFO is empty. In the example, T3 pauses on a blocking read from FIFO 2, which will not have data until T2 writes to it at cycle c-10. An exception is a thread that performs only blocking writes, such as T4. These threads assume infinite FIFO depth and do not pause, but they still contribute events to the simulation.

Each time a Func Sim thread is activated, the (F) **Task Tracker** is incremented, and when a thread pauses, it is decremented. When the Task Tracker reaches zero (i.e., all Func Sim threads are paused), the Perf Sim thread begins resolving queries.

4 Attempt to resolve queries—by the Perf Sim thread. All queries in the query pool are examined by the Perf Sim thread. For each query, the thread checks the FIFO read/write table to determine whether the corresponding NB access can proceed. Unlike LightningSim, OmniSim must consider hardware constraints, particularly FIFO sizes, to process NB FIFO accesses.

Each query has a *source* and a *target*: The *source* is the NB event being queried (e.g., the w^{th} write or r^{th} read), and the *target* is the

Query Type	Source	Target	Resolution Condition
NB Write	w^{th} write	N/A	true if $w \leq S$
NB Write	w^{th} write	$(w - S)^{\text{th}}$ read	true if source is after target
NB Read	r^{th} read	r^{th} write	true if source is after target

Table 2: Resolution for NB queries, assuming FIFO size S .

committed FIFO access against which the source must be compared. Let the FIFO size be S ; the resolution logic is shown in Table 2. Specifically, if $w \leq S$, then the NB write is guaranteed to succeed. If $w > S$, the thread checks whether the $(w - S)^{\text{th}}$ read has already occurred. Similarly, an r^{th} NB read is allowed only if the r^{th} write has already completed.

If the target event has already been simulated, the query can be resolved by comparing the actual hardware cycle values, as described in Sec. 5, which are maintained using the partial simulation graph. If the target has not yet been simulated, i.e., being unknown, the query remains in the pool and will be revisited in the next Perf Sim iteration. This is possible because of arbitrary OS scheduling.

5 Func Sim threads resume. Each time a query is resolved, the result is sent to the waiting Func Sim thread, which resumes execution immediately. The task tracker is updated accordingly. Some queries also have follow-up effects, depending on the result. Specifically, if a NB read succeeds, the FIFO is read immediately and the simulation graph is updated. If a NB write succeeds, it may unblock a pending blocking read in another thread. For example, in Fig. 7, a blocking read by T3 is automatically unpaused after a successful NB write by T2.

Finalization. Once all Func Sim threads finish execution, simulation is complete. The final simulation graph is now fully constructed. We traverse the entire graph one more time to compute the total simulation cycle count, including accurate timing for previous blocking writes and other delayed events.

7 OmniSim Optimizations

In addition to the flexibly coupled Func Sim and Perf Sim technique introduced in Sec. 6.2, OmniSim incorporates several additional optimizations that are either essential to its core simulation capabilities or critical to its efficiency.

7.1 Deadlock Detection

A critical requirement to simulate dataflow designs is to correctly detect deadlocks, which are especially common due to cyclic dependencies and improperly sized FIFOs.

First, a fundamental challenge shared by all simulators is distinguishing true, design-induced deadlocks from temporary stalls, without causing the simulator itself to hang infinitely. To address this, OmniSim employs the **(F) Task Tracker**, described in Sec. 6.2, which tracks the number of active Func Sim threads currently executing HLS code, i.e., those not waiting for a FIFO read or write. When this counter reaches zero, the Perf Sim thread checks for unresolved queries. If no queries remain, it indicates that all Func Sim threads are blocked—either on empty FIFO reads or full FIFO writes. In such cases, OmniSim immediately detects and reports a true design-level deadlock.

Second, OmniSim faces another challenge: when the Perf Sim thread is unable to resolve any pending queries because all target events remain unknown, simply returning control to the Func Sim threads would deadlock the simulator itself, as no thread can proceed. To handle this, OmniSim leverages a key insight: this situation implies that all threads have progressed to at least the clock cycle of the earliest unresolved query.⁷ Since the target event of the earliest query is still unknown, it must lie in the future relative to the query. Therefore, that query can be safely resolved as `false` (e.g., the target FIFO read has not yet occurred, so the NB write fails; similar for vice versa). By resolving the earliest query, it enables at least one Func Sim thread to resume execution, ensuring forward progress and avoiding simulator-level deadlock.

7.2 Incremental Simulation

Optimizing FIFO sizes in dataflow designs is crucial for both performance and hardware resource: smaller FIFOs may introduce stalls or even deadlocks, while larger FIFOs consume more area. In traditional HLS flows, optimizing FIFO sizes is notoriously difficult—every modification requires re-running slow C/RTL co-sim to accurately assess the performance impact.

A key advantage of LightningSim is its support for highly efficient incremental simulation when FIFO sizes change, as illustrated in Fig. 6 (top). In such cases, LightningSim can update performance estimates in just a few milliseconds during Phase 2. This is enabled by its decoupled functionality and performance simulations: the same simulation graph is reused, and only the longest-path analysis is rerun, without reconstructing the graph. For Type A designs, where functional behavior is independent of FIFO depths, the simulation graph remains valid across all FIFO configurations.

However, OmniSim builds its simulation graph dynamically during execution, based on the specific FIFO sizes used. This raises a key question: given a new set of FIFO sizes, can the existing simulation graph be reused, or is a full re-simulation required?

To address this, we introduce the concept of **constraints**. After each query is resolved as either true or false, OmniSim records the outcome as a constraint associated with the original query in the simulation graph. When a new FIFO configuration is provided,

⁷Some threads may be stalled on FIFO reads rather than queries. However, since no other task can provide writes to unblock these reads, they must remain paused until at least the time of the earliest query.

OmniSim validates the feasibility of incremental simulation by re-running the **Finalization** step (Sec. 6.2) under the new depths to compute updated cycle counts. Each stored constraint is then re-evaluated against the new results. If any query now resolves differently, for example, a NB access that previously succeeded would now fail, it indicates that the control or data flow may diverge from the original execution. In such cases, the existing simulation graph is no longer valid, and a full re-simulation is required. Otherwise, the graph can be reused to incrementally compute new cycle counts.

7.3 Runtime Optimizations

7.3.1 Graph structure. In LightningSimV2, the simulation graph is generated and stored in the compressed sparse row (CSR) format, under the assumption that it will not be traversed until fully constructed. While CSR is efficient for traversal, it relies on complex intermediate data structures to hold node and event information that has not yet been committed to the graph.

However, OmniSim requires frequent traversal of the partial simulation graph to resolve queries. To avoid copying the graph and committing all intermediate data at each evaluation, which is prohibitively expensive, OmniSim adopts an adjacency list representation for efficient construction and traversal. We specialize the format to store one edge alongside each node, minimizing pointer chasing. We also prune unnecessary nodes to reduce overhead. This design ensures that OmniSim can perform zero-copy traversal of the incomplete graph, without requiring temporary modifications.

7.3.2 Eliminating redundant FIFO checks. Evaluating non-blocking FIFO checks like `empty()` and `full()` can incur unnecessary runtime overhead, especially if their return values are never used. To address this, OmniSim introduces an LLVM pass to identify calls to such functions where the result is unused, replacing them with special markers indicating that the check can safely be skipped.

8 Experiments

In this section, we evaluate the two advantages of OmniSim: its ability to extend C-level HLS simulation to support complex dataflow designs, and its significantly improved simulation speed over state-of-the-art simulators.

First, we develop a set of **Type B** and **Type C** designs using Vitis HLS—designs that existing tools fail to simulate accurately at the C level (Sec. 3.2.3)—and show that OmniSim can correctly perform both functionality and performance simulations.

Second, we evaluate the speed of OmniSim, comparing it against co-simulation for the Type B and C designs, as well as *directly against* the state-of-the-art HLS simulator LightningSimV2 [20] on its own benchmark suite, which consists entirely of Type A designs.

All experiments are conducted on a server with a 64-core Intel Xeon Gold 6226R x86-64 CPU and 502 GiB of RAM, running Red Hat Enterprise Linux Server 8.10 and Xilinx Vitis HLS 2021.1.

8.1 Simulating Complicated Dataflow Designs

8.1.1 Design Dataset. Because of the limitations of HLS in simulating Type B and Type C designs, unfortunately, designers tend not to use HLS in these cases, making it hard to find real-world open-source examples of such designs. To the best of our knowledge, no existing HLS benchmark suite includes *any* Type B or Type C

	C-sim	Co-sim	OmniSim
fig4_ex2	@E Simulation failed: SIGSEGV.	sum_out = 2051325	sum_out = 2051325
fig4_ex3	WARNING1 (*2025); WARNING2; sum = 0	sum = 4098600	sum = 4098600
fig4_ex4a	sum_out = 2051325	sum_out = 684453	sum_out = 684453
fig4_ex4a_d	@E Simulation failed: SIGSEGV.	sum_out = 684453	sum_out = 684453
fig4_ex4b	sum_out = 2051325; Dropped = 0	sum_out = 684453; Dropped = 1348	sum_out = 684453; Dropped = 1348
fig4_ex4b_d	@E Simulation failed: SIGSEGV.	sum_out = 684453; Dropped = 1358	sum_out = 684453; Dropped = 1358
fig4_ex5	processed_by_P1 = 2025; processed_by_P2 = 0 sum_out_P1 = 2051325; sum_out_P2 = 0	processed_by_P1 = 1351; processed_by_P2 = 674 sum_out_P1 = 1366878; sum_out_P2 = 684447	processed_by_P1 = 1351; processed_by_P2 = 674 sum_out_P1 = 1366878; sum_out_P2 = 684447
fig2_timer	WARNING1 (*2025); WARNING2 Internal timer counted 0 cycles	Internal timer counted 6075 cycles	Internal timer counted 6075 cycles
deadlock	WARNING1 (*2025); WARNING2; sum = 0	ERROR!!! DEADLOCK DETECTED at 5145000 ns! SIMULATION WILL BE STOPPED!	thread 'omnisim' panicked at [...]: unresolvable deadlock detected
branch	WARNING2; fetched = 2025; executed = 102	fetched = 955; executed = 51	fetched = 955; executed = 51
multicore	WARNING2 (*16) total_fetched = 32400; total_executed = 1221	total_fetched = 15519 total_executed = 614	total_fetched = 15519 total_executed = 614
	WARNING1: Hls::stream [...] is read while empty, [...]	WARNING2: Hls::stream [...] contains leftover data, [...]	

Table 3: Comparison of Func Sim across C-sim, Co-sim, and OmniSim for Type B and Type C designs. C-sim fails to correctly simulate the functionality of these designs, whereas OmniSim matches Co-sim exactly.

Name	Type	#Mod	#FIFO	B/NB	Cyclic?	Description
fig4_ex2	B	3	2	NB	Yes	NB FIFO access (done signal)
fig4_ex3	B	3	2	B	Yes	Cyclic dependency
fig4_ex4a	C	3	1	NB	No	Skip if FIFO full
fig4_ex4a_d	C	3	2	NB	Yes	Skip if full (done signal)
fig4_ex4b	C	3	1	NB	No	Count dropped elements
fig4_ex4b_d	C	3	2	NB	Yes	Count dropped (done signal)
fig4_ex5	C	4	2	NB	No	Congestion-aware select
fig2_timer	C	3	2	NB	Yes	Fixed-point cycle count
deadlock	B	3	2	B	Yes	Mutual blocking read
branch	C	3	2	NB	Yes	Branch instructions
multicore	C	34	64	NB	Yes	Multiple cores with branches

Table 4: Evaluated Type B and Type C designs.

designs. Our dataset of Type B and C designs therefore consists mostly of smaller examples; however, these examples are only used for demonstration and do not limit OmniSim’s generality.

We enumerate these designs in Table 4.⁸ First, we include all the Type B and Type C designs shown in Fig. 4, including non-blocking accesses in an infinite loop (terminated by a done signal), cyclic dependencies, and cycle-dependent functionality variance. We also implement the timer pattern shown in Fig. 2, another Type C design, to test OmniSim’s ability to orchestrate threads independent of OS thread scheduling. We test OmniSim’s deadlock detection capability with a cyclic dataflow designed to deadlock, in which two tasks block trying to read empty FIFOs. We also include a design called branch where a downstream executor task influences an upstream data fetcher, and a larger multicore version.

8.1.2 Functionality Simulation. We compare the outputs of each design using different simulation methods in Table 3. Clearly, *none can be correctly simulated by C-sim*. For instance, three of our designs (fig4_ex2, fig4_ex4a_d, and fig4_ex4b_d) contain infinite loops reading the input array in the producer task that are terminated by a done signal from the consumer task. In C-sim, *these designs crash* because they run off the end of the input array but never proceed to simulate the consumer task that sends the done signal. Even designs that do not fail so catastrophically still exhibit strange and inaccurate behavior under C-sim. For instance, fig4_ex3 emits many warnings indicating that a stream “is read while empty” or “contains leftover data”, and outputs zero in place

⁸Designs available at <https://github.com/sharc-lab/omnisim-benchmarks>.

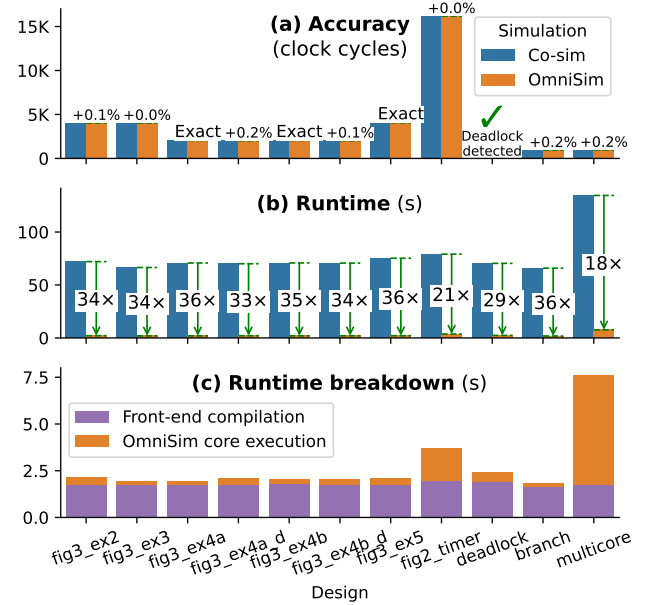


Figure 8: Comparisons of OmniSim’s (a) cycle-accuracy and (b) simulation runtime against co-sim. A more detailed breakdown of the OmniSim execution time is shown in (c).

of the expected sum. Meanwhile, other designs—like fig4_ex4a, fig4_ex4b, and fig4_ex5—silently give incorrect results based on the wrong assumption that write_nb() calls always succeed.

In contrast, the output of each design under OmniSim matches the expected output produced by co-sim in every case—including producing the correct timer output in fig2_timer (6075 cycles), as well as detecting the deadlock in the deadlock design.

8.1.3 Performance Simulation. OmniSim also obtains highly accurate cycle counts for each of these hard-to-simulate designs, closely aligning with the cycle counts produced by C/RTL co-simulation. Results are shown in Fig. 8(a). We observe cycle counts that are, on average, only **0.09% off** from the cycle counts reported by co-simulation. Even though LightningSim cannot support these designs, we benefit from the high accuracy of LightningSim’s simulation graph approach atop which OmniSim was developed.

Benchmark	LSv2	OmniSim			$\Delta \downarrow$
	Total	Total	FE	MT	
Fixed-point square root [22]	4.97	3.65	3.42	0.23	1.36×
FIR filter [22]	2.43	1.94	1.74	0.20	1.25×
Fixed-point window conv [22]	3.69	3.15	2.77	0.38	1.17×
Floating point conv [22]	2.42	2.46	2.11	0.35	0.98×
Arbitrary precision ALU [22]	2.12	2.03	1.81	0.22	1.04×
Parallel loops [22]	2.34	2.16	1.94	0.23	1.08×
Imperfect loops [22]	2.24	2.13	1.91	0.23	1.05×
Loop with max bound [22]	2.25	2.14	1.91	0.23	1.05×
Perfect nested loops [22]	2.27	2.12	1.90	0.22	1.07×
Pipelined nested loops [22]	2.23	2.19	1.96	0.23	1.02×
Sequential accumulators [22]	2.29	2.20	1.98	0.22	1.04×
Accumulators + asserts [22]	2.30	2.30	2.08	0.22	1.00×
Accumulators + dataflow [22]	2.29	2.19	1.95	0.24	1.05×
Static memory example [22]	2.18	2.12	1.92	0.19	1.03×
Pointer casting example [22]	2.15	2.13	1.94	0.19	1.01×
Double pointer example [22]	2.14	1.91	1.71	0.19	1.12×
AXI4 master [22]	2.19	2.07	1.90	0.17	1.06×
AXIS w/o side channel [22]	2.06	1.94	1.77	0.17	1.06×
Multiple array access [22]	2.18	2.08	1.87	0.21	1.05×
Resolved array access [22]	2.20	2.05	1.84	0.21	1.07×
URAM with ECC [22]	2.21	2.05	1.85	0.19	1.08×
Fixed-point Hamming [22]	2.37	2.46	2.13	0.33	0.96×
Unoptimized FFT [13]	2.78	2.91	2.28	0.63	0.95×
Multi-stage FFT [13]	2.67	2.93	2.29	0.64	0.91×
Huffman encoding [13]	2.63	2.32	1.99	0.33	1.13×
Matrix multiplication [13]	2.61	2.59	2.38	0.21	1.01×
Parallelized merge sort [13]	2.27	2.15	1.96	0.18	1.06×
Vector add with stream [23]	4.48	3.56	2.17	1.39	1.26×
FlowGNN GIN [18]	28.9	11.97	8.88	3.08	2.42×
FlowGNN GCN [18]	30.9	17.18	11.38	5.80	1.80×
FlowGNN GAT [18]	41.6	24.60	20.53	4.07	1.69×
FlowGNN PNA [18]	30.5	29.00	23.30	5.70	1.05×
FlowGNN DGN [18]	26.9	11.71	6.92	4.79	2.30×
INR-Arch [4]	128.4	26.39	9.88	16.51	4.87×
SkyNet [24]	3103	469.3	7.55	461.8	6.61×

All times are indicated in seconds.

LSv2: LightningSimV2. OmniSim is further broken down into:

FE: front-end compilation (§6.1). MT: multi-threaded execution (§6.2).

 $\Delta \downarrow$: Speedup of OmniSim over LightningSimV2.

Table 5: Comparisons of OmniSim over LightningSimV2.

8.2 Simulation Speed

8.2.1 Type B and C Designs. Fig. 8(b) compares OmniSim speed against co-simulation, achieving a geomean of **30.7× speedup**. In the best case, achieving 35.9× speedup, OmniSim reduces the simulation from **66 seconds** by co-sim to only **1.83 seconds** at C level. We also emphasize that our reported times are *end-to-end*, including the time needed for front-end compilation (Sec. 6.1), which, in fact, usually dominates the majority of the OmniSim runtime (shown in purple in Fig. 8(c)).

8.2.2 Versus LightningSimV2. Improvements over co-sim are particularly valuable for designs previously unsupported by state-of-the-art simulators, while efficiency remains important. Using LightningSimV2’s own benchmark suite (Table 5), we show that OmniSim achieves a **1.26×** geomean speedup over LightningSimV2

Description	Depths	Incr.	OK?	FE	MT	Total	$\Delta \downarrow$
Initial run	(2, 2)	—	—	1.75 s	0.35 s	2.10 s	
Incremental	(2, 100)	77.86 μ s	✓	—	—	77.86 μ s	(2.7E4×)
Non-incremental	(100, 2)	85.18 μ s	✗	—	0.31 s	0.31 s	(6.77×)

Incr.: Time for incremental re-simulation. OK? if it satisfied constraints (§7.2).

FE: front-end compilation (§6.1). MT: multi-threaded execution (§6.2).

 $\Delta \downarrow$: Speedup over full simulation.

Table 6: Evaluating fig4_ex5 under different FIFO depths.

with no loss of accuracy. Minor slowdowns observed in four designs (the largest from 2.67 to 2.93 seconds) are likely due to random variations in the runtime environment.

We draw particular attention to the largest benchmarks listed last in the table, INR-Arch and SkyNet. OmniSim reduces the simulation time for INR-Arch by **4.87×**, from over 2 minutes to 26.4 seconds, and it reduces SkyNet’s simulation time by **6.61×** from over 51 minutes to under 8 minutes. Since these Type A designs contain no non-blocking calls and thus must never wait for simulated timing feedback from the shared Perf Sim thread, they observe pure performance gains from our multithreaded architecture. These results show that our flexibly coupled simulation is *not a compromise* over LightningSim’s decoupled architecture but actually *improves simulation performance*, especially on the large designs.

8.2.3 Incremental Simulation. Like LightningSim, OmniSim features the capability to *incrementally* re-simulate designs with changed FIFO buffer depths. To evaluate this capability, we perform a case study on fig4_ex5, a Type C design which routes requests between two PEs based on FIFO backpressure.

Results are shown in Table 6. After the initial run, we re-simulate with each of the two FIFO depths set to 100. In the first case, incremental re-simulation completes in just 78 μ s, achieving a **26,966×** speedup over full simulation. In the second case, incremental simulation fails due to violated *constraints* (Sec. 7.2), as the HLS code would have behaved differently under this configuration, requiring a full re-simulation. Even so, reusing the already-compiled executable yields a 6.77× speedup.

9 Conclusion

In this work, we proposed OmniSim, a novel solution that addresses the simulation limitations of existing HLS tools by bridging the gap between C-level abstractions and hardware-level semantics. OmniSim achieves near-C simulation speed while maintaining near-RTL accuracy for both functionality and performance simulation, even for complex dataflow designs. We first introduced a new taxonomy and conducted an in-depth analysis of dataflow patterns that are particularly challenging to simulate. To tackle these challenges, OmniSim carefully orchestrates functionality and performance simulation threads to accurately model hardware-level behavior under arbitrary OS scheduling. We then demonstrated that OmniSim can successfully simulate eleven designs that were unsupported by existing HLS tools, achieving an average speedup of 32.1× over RTL simulation. We further showed that OmniSim provides substantial simulation speedup—up to 6.61×—compared to the state-of-the-art simulator for large-scale designs. Overall, OmniSim enables more agile, productive, and realistic hardware development workflows, bringing HLS significantly closer to being a true RTL replacement.

References

- [1] [n. d.]. Verilator. <https://www.veripool.org/verilator/>.
- [2] Mohammed Abderehman, Jayprakash Patidar, Jay Oza, Yom Nigam, Tm Abdul Khader, and Chandan Karfa. 2022. FastSim: A Fast Simulation Framework for High-Level Synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 5 (May 2022), 1371–1385. doi:10.1109/TCAD.2021.3090339
- [3] Stefan Abi-Karam, Rishov Sarkar, Dejie Xu, Zhiwen Fan, Zhangyang Wang, and Cong Hao. 2023. INR-Arch: A dataflow architecture and compiler for arbitrary-order gradient computations in implicit neural representation processing. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [4] Stefan Abi-Karam, Rishov Sarkar, Dejie Xu, Zhiwen Fan, Zhangyang Wang, and Cong Hao. 2023. INR-Arch: A Dataflow Architecture and Compiler for Arbitrary-Order Gradient Computations in Implicit Neural Representation Processing. In *Proceedings of the 42nd IEEE/ACM International Conference on Computer-Aided Design (ICCAD '23)*. Association for Computing Machinery, San Francisco, CA, USA.
- [5] AMD. [n. d.]. Vitis High-Level Synthesis. <https://www.xilinx.com/products/design-tools/vivado/high-level-design.html>.
- [6] Suhail Basalama and Jason Cong. 2025. Stream-HLS: Towards Automatic Dataflow Acceleration. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. 103–114.
- [7] Andrew Canis, Jongsok Choi, Mark Aldham, Victor Zhang, Ahmed Kammoona, Jason H Anderson, Stephen Brown, and Tomasz Czajkowski. 2011. LegUp: high-level synthesis for FPGA-based processor/accelerator systems. In *Proceedings of the 19th ACM/SIGDA international symposium on Field programmable gate arrays*. 33–36.
- [8] Hongzheng Chen, Niansong Zhang, Shaojie Xiang, Zhichen Zeng, Mengjia Dai, and Zhiru Zhang. 2024. Allo: A programming model for composable accelerator design. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 593–620.
- [9] Young-Kyu Choi, Yuze Chi, Jie Wang, and Jason Cong. 2020. FLASH: Fast, Parallel, and Accurate Simulator for HLS. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 12 (Dec. 2020), 4828–4841. doi:10.1109/TCAD.2020.2970597
- [10] Fabrizio Ferrandi, Vito Giovanni Castellana, Serena Curzel, Pietro Fezzardi, Michele Fiorito, Marco Lattuada, Marco Minutoli, Christian Pilato, and Antonino Tumeo. 2021. Invited: Bambu: an Open-Source Research Framework for the High-Level Synthesis of Complex Applications. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1327–1330. doi:10.1109/DAC18074.2021.9586110
- [11] Licheng Guo, Yuze Chi, Jason Lau, Linghao Song, Xingyu Tian, Moazin Khatti, Weikang Qiao, Jie Wang, Ecenur Ustun, Zhenman Fang, Zhiru Zhang, and Jason Cong. 2023. TAPA: A Scalable Task-Parallel Dataflow Programming Framework for Modern FPGAs with Co-Optimization of HLS and Physical Design. *ACM Trans. Reconfigurable Technol. Syst.* 16, 4 (Dec. 2023), 63:1–63:31. doi:10.1145/3609335
- [12] Sagar Karandikar, Howard Mao, Donggyu Kim, David Biancolin, Alon Amid, Dayeol Lee, Nathan Pemberton, Emmanuel Amaro, Colin Schmidt, Aditya Chopra, et al. 2018. FireSim: FPGA-accelerated cycle-exact scale-out system simulation in the public cloud. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 29–42.
- [13] Ryan Kastner, Janarbek Matai, and Stephen Neuendorffer. 2018. Parallel Programming for FPGAs. arXiv:1805.03648 [cs] doi:10.48550/arXiv.1805.03648
- [14] Youngsoo Kim, Shrikant Jadhav, and Clay S Gloster. 2016. Dataflow to hardware synthesis framework on fpgas. In *2016 International Symposium on Computer Architecture and High Performance Computing Workshops (SBAC-PADW)*. IEEE, 91–96.
- [15] Alberto Lerner and Gustavo Alonso. 2024. Data flow architectures for data processing on modern hardware. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 5511–5522.
- [16] Tony Nowatzki, Vinay Gangadhar, Newsha Ardalani, and Karthikeyan Sankaralingam. 2017. Stream-dataflow acceleration. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 416–429.
- [17] Rishov Sarkar, Stefan Abi-Karam, Yuqi He, Lakshmi Sathidevi, and Cong Hao. 2023. FlowGNN: A dataflow architecture for real-time workload-agnostic graph neural network inference. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1099–1112.
- [18] Rishov Sarkar, Stefan Abi-Karam, Yuqi He, Lakshmi Sathidevi, and Cong Hao. 2023. FlowGNN: A Dataflow Architecture for Real-Time Workload-Agnostic Graph Neural Network Inference. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, Montreal, QC, Canada, 1099–1112. doi:10.1109/HPCA56546.2023.10071015
- [19] Rishov Sarkar and Cong Hao. 2023. LightningSim: Fast and Accurate Trace-Based Simulation for High-Level Synthesis. In *2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Marina Del Rey, CA, USA, 1–11. doi:10.1109/FCCM57271.2023.00010
- [20] Rishov Sarkar, Rachel Paul, and Cong Hao. 2024. LightningSimV2: Faster and Scalable Simulation for High-Level Synthesis via Graph Compilation and Optimization. In *2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Orlando, FL, USA, 104–114. doi:10.1109/FCCM60383.2024.00021
- [21] Siemens. [n. d.]. Catapult High-Level Synthesis and Verification. <https://eda.sw.siemens.com/en-US/ic/catapult-high-level-synthesis/>.
- [22] Xilinx. 2021. Basic Examples for Vitis HLS. GitHub.
- [23] Xilinx. 2022. Vitis Accel Examples' Repository. GitHub.
- [24] Xiaofan Zhang, Haoming Lu, Cong Hao, Jiachen Li, Bowen Cheng, Yuhong Li, Kyle Rupnow, Jinjun Xiong, Thomas Huang, Honghui Shi, Wen-Mei Hwu, and Deming Chen. 2020. SkyNet: A Hardware-Efficient Method for Object Detection and Tracking on Embedded Systems. *Proceedings of Machine Learning and Systems* 2 (March 2020), 216–229.
- [25] Chenfeng Zhao, Clayton J. Faber, Roger D. Chamberlain, and Xuan Zhang. 2024. HLPf: Demystifying the Performance of HLS-based Graph Neural Networks with Dataflow Architectures. *ACM Trans. Reconfigurable Technol. Syst.* (April 2024). doi:10.1145/3655627